CIS 678 - Machine Learning

– Ensemble Methods –

Instructor: Yong Zhuang

yong.zhuang@gvsu.edu

Last Class

- Boosting combines weak learners into a stronger classifier.
- AdaBoost adjusts example weights to emphasize misclassified examples.
- The final prediction uses a weighted vote of weak learners.
- Training error can decrease rapidly when each learner has an advantage over random guessing.
- Margins help explain generalization beyond training accuracy.

Agenda

- Ensemble learning: why combine models?
- Bagging and random forests
- Boosting and AdaBoost
- Gradient boosting and XGBoost
- Handling class imbalance

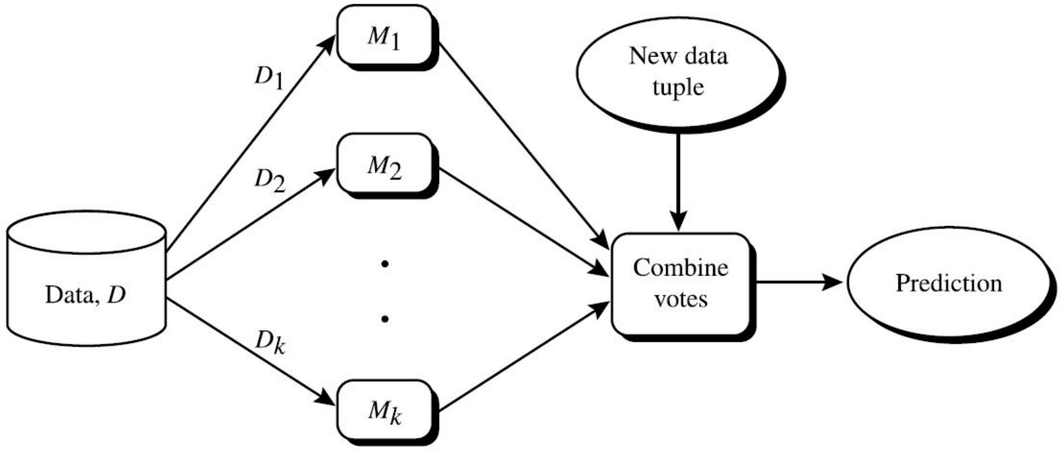
Ensemble Learning

Combine predictions from multiple models to build a stronger predictor.

- **Why it can help:** Models can make different errors, so combining their predictions can improve performance.
- **Bagging:** Train models on bootstrap samples, then combine predictions by voting or averaging.
- **Boosting:** Build models sequentially, with each new model addressing weaknesses of the current ensemble.
- **Voting ensembles:** Combine predictions from different models, such as a decision tree, logistic regression, and a neural network.



How do we create different models? How do we combine their predictions?



Bagging: Bootstrap Aggregation

Main idea: Train models on different bootstrap samples and combine their predictions.

1. From a training set of (n) examples, draw (n) examples **with replacement**.
2. Train a model on this bootstrap sample.
3. Repeat to build multiple models.

Original: A, B, C, D
Bootstrap samples: A, A, C, D / B, C, C, D

Combine predictions

- Classification: majority vote
- Regression: average prediction

Why it helps

- Often reduces variance, especially for deep decision trees.
- Improvement depends on how different the models' errors are.

Boosting: Learning Sequentially

Main idea: Build an ensemble step by step, with each new learner addressing weaknesses of the current ensemble.

AdaBoost's approach

- Train a weak classifier using weighted training examples.
- Increase the relative weight of examples it misclassifies.
- Train the next classifier using the updated weights.
- Combine classifiers through a weighted vote based on their weighted training errors.

Compared with bagging

- Bagging trains models independently on bootstrap samples.
- Boosting trains models sequentially.
- Boosting can improve performance, but emphasizing difficult examples can make it sensitive to mislabeled data.

AdaBoost: Binary Classification

Given n training examples (x_j, y_j), with $y_j \in \{-1, +1\}$:

Initialize: $w_j^{(1)} = 1/n$.

At each round t:

1. Train a weak classifier h_t using the current example weights.
2. Compute its weighted error on the original training set:
   $\epsilon_t = \sum_{j=1}^{n} w_j^{(t)} \mathbf{1}[h_t(x_j) \neq y_j]$.
3. Set its voting weight: $\alpha_t = \frac{1}{2} \ln \frac{1-\epsilon_t}{\epsilon_t}$.
4. Update and normalize the example weights: $w_j^{(t+1)} = \frac{w_j^{(t)} e^{-\alpha_t y_j h_t(x_j)}}{Z_t}$.

Final prediction:

$$H(x) = \text{sign}\left(\sum_{t=1}^{T} \alpha_t h_t(x) \right).$$

Gradient Boosting: Correcting Prediction Errors

Main idea: Add a model at each round to reduce the current ensemble's loss.

For regression with squared-error loss:

$$L(y_i, F(x_i)) = \frac{1}{2}(y_i - F(x_i))^2$$

1. **Initialize with the mean target:** $F_0(x) = \bar{y}$.
2. **Compute residuals at round t:** $r_{it} = y_i - F_{t-1}(x_i)$.
3. **Fit a regression tree** M_t to the examples (x_i, r_{it}).
4. **Update the ensemble:**

$$F_t(x) = F_{t-1}(x) + \eta M_t(x),$$

where η is the learning rate.

For other differentiable losses, fit the new learner to **negative gradients**.

Stop after a chosen number of rounds or use validation-based early stopping.

XGBoost: Efficient, Regularized Boosted Trees

XGBoost is a widely used implementation of gradient-boosted trees for classification and regression.

- **Regularization:** Penalizes tree complexity and leaf weights to help control overfitting.
- **Loss-based optimization:** Uses first- and second-order derivatives to choose useful tree updates.
- **Efficient training:** Uses optimized algorithms for finding splits and building trees.

Practical controls: Learning rate, tree depth, subsampling, and validation-based early stopping.

Random Forests: Bagging with Feature Randomness

Main idea: Combine many decision trees while reducing how similarly they behave.

Training

- Train each tree on a bootstrap sample of the training data.
- At each node, randomly select a subset of features.
- Choose the best split using only those candidate features.
- Repeat to build many trees.

Key settings: Number of trees, features considered per split, and tree depth or minimum leaf size.

Prediction

- Classification: combine trees by voting or averaging class probabilities.
- Regression: average the trees' predictions.

Why it helps

- Feature randomness can reduce correlation between trees.
- Combining trees reduces variance compared with a single deep tree.

Practical Consideration: Class Imbalance

Problem: One class has far fewer examples than another.

Example: If only 1% of transactions are fraudulent, predicting “not fraud” for every transaction gives 99% accuracy but detects no fraud.

Possible approaches

- **Class weighting:** Give minority-class errors greater importance during training.
- **Resampling:** Oversample the minority class or undersample the majority class.
- **Threshold adjustment:** Increase minority-class recall while accepting more false positives.
- **Ensemble adaptations:** Use balanced sampling or class weighting when training ensemble members.

Evaluate carefully

- Examine precision, recall, and the confusion matrix for each class.
- Resample training data only. Keep validation and test data representative of the intended application.

Ensemble Methods for Class Imbalance

Ensemble method	Strategy for imbalance	Important limitation
Bagging	Train models on balanced samples	Ordinary bootstrap sampling preserves imbalance on average
Random forests	Use balanced sampling or class weights	Standard forests can still favor the majority class
AdaBoost	Use balanced sampling or a cost-sensitive variant	Emphasizing mistakes does not explicitly prioritize the minority class
Gradient boosting / XGBoost	Give minority-class examples greater weight in the loss	Larger weights may increase false positives

Validate weights and thresholds using minority-class precision and recall. Resample training data only.

Summary

- **Ensemble learning** combines multiple models and can improve predictions when their errors differ.
- **Bagging** trains models on bootstrap samples and combines predictions by voting or averaging to reduce variance.
- **Random forests** add random feature selection at each split to reduce correlation between trees.
- **Boosting** builds models sequentially, with each new learner addressing weaknesses of the current ensemble.
- **AdaBoost** adjusts training-example weights and combines weak classifiers through a weighted vote.
- **Gradient boosting** fits new learners to negative gradients of the loss. **XGBoost** adds regularization and efficient optimization.
- **Class imbalance** requires suitable weighting, sampling, or threshold adjustment, with evaluation beyond overall accuracy.